

Agenda

- MPA vs SPA
- React
- JSX
- React Component

Multi page application (MPA)

- the application which contains multiple pages designed
- it sends the request to the server every time for a page when data is requested
- Executed only on the server side
- requires more resources compared with SPA
- e.g. MPA (amazon.in)

Single page application (SPA)

- the application which has only single html page
- Gets loaded only once when user visits the website
- Once loaded, it sends the request to the server only to get the data
- Executed only on the client side (inside a browser)
- requires less resources compared with MPA
- it has only one html page named index.html
- e.g. react, angular, vue

Advantages of SPA

- SPA application can cache the contents for offline use
- faster than multi-page application
- provides component-based architecture which promotes the re-usability

Disadvantages of SPA

- the first time loading is bit slower than multi-page application
- it has to load all the components(sections) while loading the application for the first time
- these applications are memory-heavy

Frameworks or libraries used to develop SPAs

1. react: developed by Facebook in 2013
2. angular: developed by Google in 2011
3. preact: light weight react
4. svelte: faster than react but has less ecosystem

tooling for any framework

- language: react (JS), angular (TS), preact (JS), svelte (JS)
- bundler:

- tool to create a bundle (archive of all the required pages of a web application)
- e.g. Webpack, vite
- platform: node.js

Angular

- angular is a framework used to develop SPA type application
- developed by Google in 2011
- developed in TypeScript
- TS is required to develop angular application
- bit slower than react as it requires more memory
- framework
 - contains different modules/libraries
 - most of the times, a frame is built using multiple languages
 - e.g. angular: TS + HTML + CSS + JS + others

What is React

- It is a JS library used to develop Single Page Application
- React is used to develop client side applications
- React features
 - Has a component-driven architecture
 - Uses virtual DOM for improving the application performance
 - Eco-system: React Router, React Redux Toolkit, React Native
- since it is a library, it requires less memory compared to Angular
- it is bit faster than Angular

History

- Created by: Jordan Walke at Facebook (2011)
- First used: Facebook News Feed (2011)
- Open-sourced: May 2013 (React Conf 2015 marked major adoption)
- Current maintainers: Meta (Facebook) + community (OpenJS Foundation)

Virtual DOM

- in-memory copy of browser or real DOM
- real DOM is created by the browser to render the UI in the browser's window
- when real DOM changes, the entire real DOM gets re-rendered
- this is very memory-heavy operation which may slow down the UI generation
- react create a in-memory of real DOM called as virtual DOM to solve the re-rendering of web page when there is a change in the page
- how virtual DOM works
 - when react application loads, react library creates a copy of real DOM and renders the entire web page when it gets loaded initially (only for the first time)
 - when there is a change required, the change will be performed on the virtual DOM first
 - it will updates the virtual DOM with new copy of changed element
 - then react finds the difference between old DOM object and new DOM object (diffing algorithm)

- then react creates a patch of new changes to be applied on the real DOM
- finally the patch will be applied on real DOM to change the browser UI (re-conciliation)
- because of this process, the entire real DOM never reloads which is why the react application is faster than MPA

CDN links used for react

```
<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js">
</script>
<!-- Don't use this in production: -->
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
```

- `react.development.js`
 - used for developing the react application
 - used to create react components
- `react-dom.development.js`
 - used to render the react components inside the browser
- `babel.min.js`
 - used to convert the JSX code into JS code

Pure React (Without JSX)

Core Concepts

- Direct manipulation of Virtual DOM using React's JavaScript API
- Uses `React.createElement()` to construct UI elements
- Manual DOM mounting with `ReactDOM.createRoot()`

Example

```
<!DOCTYPE html>
<html>
  <head>
    <script
      crossorigin
      src="https://unpkg.com/react@18/umd/react.development.js"
    ></script>
    <script
      crossorigin
      src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
    ></script>
  </head>
  <body>
    <div id="root1"></div>
    <div id="root2"></div>

    <script>
```

```
// Simple element creation
const root1 = document.getElementById("root1");
const h1 = React.createElement("h1", null, "Hello React");
ReactDOM.render(h1, root1); //(prior react 18)

// Complex element hierarchy
const root2 = ReactDOM.createRoot(document.getElementById("root2"));
const h4 = React.createElement(
  "h4",
  { key: "h4" },
  "The library for web and native user interfaces"
);
const p = React.createElement(
  "p",
  { key: "p" },
  "React is a free and open-source front-end JavaScript library that
  aims to make building user interfaces based on components more seamless."
);
const container = React.createElement("div", null, [h4, p]);
root2.render(container); //(from react 18)

</script>
</body>
</html>
```

Key Observations in above example

1. Verbose Syntax: Each element requires explicit `createElement` call
2. Manual Structure: Nested elements become hard to visualize
3. No Components: Just direct Virtual DOM node creation
4. Performance: Still uses Virtual DOM diffing

Render single element(From React 18)

```
const root = document.getElementById('root')
const element = React.createElement('h2', {}, 'welcome to react')
ReactDOM.createRoot(root).render(element)
```

Render multiple elements

```
// get root element
const root = document.getElementById('root')
// create required element(s)
const h1 = React.createElement('h1', {}, 'this is a header1')
const h2 = React.createElement('h2', {}, 'this is a header2')
const h3 = React.createElement('h3', {}, 'this is a header3')
// render the elements by using an array of elements
ReactDOM.createRoot(root).render([h1, h2, h3])
```

Render hierarchy of element

```
// create root element
const root = document.getElementById('root')
// create a child element
const h1 = React.createElement('h1', {}, 'Child element of div')
// create parent div
const div = React.createElement('div', {}, h1)
// render the element
ReactDOM.createRoot(root).render(div)
```

React (Using Babel + JSX)

Babel's Role

- It is a JavaScript compiler used to compile one form of JS to another
- Converts modern JS/JSX to browser-compatible code
- used to convert higher version (which browser can NOT understand) of JS to lower version (which browser can understand)
- JSX Transformation: Converts HTML-like syntax to `React.createElement()` calls
- can be added in a page using CDN link

Example

```
<!DOCTYPE html>
<html>
  <head>
    <script
      crossorigin
      src="https://unpkg.com/react@18/umd/react.development.js"
    ></script>
    <script
      crossorigin
      src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
    ></script>
    <script
      crossorigin
      src="https://unpkg.com/@babel/standalone/babel.min.js"
    ></script>
  </head>
  <body>
    <div id="root1"></div>
    <div id="root2"></div>

    <!-- Babel-processed scripts -->
    <script type="text/babel">
      // Component-based approach
      const root1 = ReactDOM.createRoot(document.getElementById("root1"));
    </script>
  </body>
</html>
```

```

    const Hdr1 = () => {
      return <h1>Hello React</h1>;
    };

    root1.render(<Hdr1 />);
  </script>

  <script type="text/babel">
    const root2 = ReactDOM.createRoot(document.getElementById("root2"));

    const Container = () => {
      return (
        <div>
          <h4>The library for web and native user interfaces</h4>
          <p>
            React is a free and open-source front-end JavaScript
library that
            aims to make building user interfaces based on
components more
            seamless.
          </p>
        </div>
      );
    };

    root2.render(<Container />);
  </script>
</body>
</html>

```

Key Improvements

1. JSX Syntax: HTML-like templating

```

// Transforms to:
// React.createElement("h1", null, "Hello React")
<h1>Hello React</h1>

```

2. Component Model: Reusable UI units

```

function MyComponent() {
  return <div>Content</div>;
}

```

3. Readability: Visual hierarchy matches DOM structure

4. Developer Experience: Faster prototyping

JSX Under the Hood

Compilation Process

- Test on Babel website (with classic runtime).
 - <https://babeljs.io/repl>

```
// Original JSX
<div className="header">
  <h1>Title</h1>
  <p>Content</p>
</div>;

// compiled to:
React.createElement(
  "div",
  { className: "header" },
  // next arguments are variable arg list that is collected as array [...]
  React.createElement("h1", null, "Title"),
  React.createElement("p", null, "Content")
);
```

React Important Concepts

1. Component Architecture

- Reusable Components

```
// Component = JavaScript function returning JSX
function Greeting() {
  return <h2>Welcome to React!</h2>;
}

// Component Composition
function App() {
  return (
    <div>
      <h1>Hello World</h1>
      <Greeting />
      <Greeting />
    </div>
  );
}
```

2. JSX Styling

- Inline styles as objects

```
<div className="container" style={{ color: "red" }}>
  <p>Styled content</p>
</div>
```

```
const style = {
  container: {
    color: "red"
  },
  content: {
    fontSize: "14px",
    color: "blue"
  }
}

<div className="container" style={style.container}>
  <p style={style.content}>Styled content</p>
</div>
```

- class vs className

```
// Correct
<div className="book-card">...</div>

// Incorrect
<div class="book-card">...</div>
```

3. Embedding JavaScript in JSX/Data binding

- Using the variable value inside a html tag
- In react, it will be done using interpolation
- use the `{}` brackets for loading the variables value inside the html tag

```
const myvar = 100
<h3>{myvar}</h3>
```

- To render an object, split the object into properties and use interpolation to render those properties

```
const jamesBond = {
  name: "James Bond",
  age: 65,
  address: "London",
  speciality: "Adventurous Spy",
};
```



```
// Variables in curly braces
<div>Name: {jamesBond.name}</div>
<div>Address: {jamesBond['address']}</div>

// Expressions work too
<div>Status: {age > 60 ? 'Veteran' : 'Active'}</div>
```

- to render an array, use the map function to iterate over the array and use interpolation to render the properties of the object

```
const heros = [
  {
    name: "Iron Man",
    age: 48,
    address: "Stark Tower, New York",
    speciality:
      "Genius-level intellect, Powered armor suit, Advanced technology",
  },
  {
    name: "Captain America",
    age: 105,
    address: "Brooklyn, New York",
    speciality: "Super strength, Enhanced agility, Vibranium shield combat",
  },
];

const MarvelHeros = () => {
  return (
    <div>
      {heros.map((hero) => (
        <div>
          <h3>Marvel Hero</h3>
          <div>Name: {hero.name}</div>
          <div>Address: {hero.address}</div>
          <div>Age: {hero.age}</div>
          <div>Speciality: {hero.speciality}</div>
        </div>
      ))}
    </div>
  );
};
export default MarvelHeros;
```

JSX Syntax understanding for

1. Self-Closing Tags

```
// Invalid

```

```
// Always required for void elements

```

2. Root Element Requirement

```
// Invalid
return (
  <h3>Title</h3>
  <p>Content</p>
)

// Valid
return (
  <div>
    <h3>Title</h3>
    <p>Content</p>
  </div>
)

// Valid (Non rendered - parent tag)
return (
  <>
    <h3>Title</h3>
    <p>Content</p>
  </>
)
```

Summary: conventions to write JSX

- use xml (html) syntax in JS
- the properties are using camelCase: className
- to read value of a variable use string interpolation ({})
- array
 - to read values of an array use map function
 - in array, every element must have a unique key to identify each element uniquely
 - these keys are used by diffing algorithm to track the changes from older to new state of virtual DOM
- execute the JS expression or function using {}

Vite

- a build tool used to build multiple types of project
- e.g. react, angular, svelte, preact etc.
- build
 - compile the project: minifying (reducing the file size) the js files
 - add the external referenced packages
 - create a bundle (collection of required files) for deployment

- create a vite project using npm

project management

```
# create a new project
# > npm create vite <application name>
> npm create vite app1
> yarn create vite app1
> npx vite app1

# go to the project
> cd app1

# install the dependencies
> npm install
> yarn

# start running the application
> npm run dev
> yarn dev

# build the application
> npm run build
> yarn build

# unit test the application
> yarn test
```

Project hierarchy

- node_modules (directory)
 - contains all the dependencies mentioned in package.json
- public (directory)
 - contains the public resources like images, audio, video etc.
- src (directory)
 - contains the application source code (components, assets etc)
 - assets (directory)
 - contains application assets like images
 - components (directory)
 - contains the reusable components
 - pages (directory)
 - contains the components which represent the website pages
 - services (directory)
 - contains the functions which connects the frontend to the backend
 - the function will call the REST APIs
 - App.css
 - contains the css rules for App component
 - App.jsx

- contains the application's startup or main component
 - the application starts with App component
- index.css
 - contains the css rules shared across the components
- main.jsx
 - contains the application startup code
- .gitignore
 - used to mention the files which need not to be added to the git repository
- eslint.config.js
 - configuration file used by ESLint
- index.html
 - only html file in the project
 - contains the application startup code
- package.json
 - contains the application dependencies, scripts and metadata
 - dependencies
 - list of modules required to run the application
 - these modules will be bundled when the app gets built
 - devDependencies
 - list of modules required to develop the application
 - these modules will NOT be bundled when the application gets built
- Readme.md
 - contains the information about the application
- vite.config.js
 - contains the vite configuration
- yarn.lock
 - contains the packages installed with the required versions
 - created when yarn is used to install the dependencies
- package-lock.json
 - contains the packages installed with the required versions
 - created when npm is used to install the dependencies

Project startup

- yarn dev or npm run dev is fired
- starts lite web server on port 5173
- the server loads the file named index.html
- index.html loads src/main.jsx
- main.jsx
 - find an object of div having root id
 - renders the first component named App (which is defined in src/App.jsx)

React Component

- Component is reusable entity which contains user interface defined in html code
- A component can be loaded using the component name as a tag (enclosed by `<` and `>`)

- It Contains -
 1. user interface: created using JSX syntax
 2. state (optional): used to store some data for internal use
 3. event handlers (optional): used to handle events generated by the component UI
 4. props (optional): parameters (properties) of a component
 5. mandatory to return only one element as a return value of a component. If you have multiple elements, then wrap them inside a parent element

- Types of components

1. Functional component

- Component created using a function
- A javascript function which returns a JSX code to create its user interface
- Before react 16, functional components were used only for stateless implementation (the component which does not require to maintain the state)
- stateless means a component which is not maintaining its state
- Since the react 16 introduced a concept called as a react hooks, it is possible now to create functional components to store the state
- Hence the class components are not need anymore and by default we use a function to create component
- it is faster than class component
- it is light weight component

```
const root = document.getElementById('root')
// functional component
function Dummy() {
  return <div>this is a functional component </div>
}
// here the <Dummy /> is function component
ReactDOM.createRoot(root).render(<Dummy />)
```

2. Class component

- Component create using a class
- Before react 16, class components were used to create stateful components (a component which can maintain its state)
- It is a class derived from React.Component class
- this class implements method named render()
- used a lot before react hooks to create stateful component
- stateful means a component which is maintaining its state for internal use
- using class component is not required as the react hook can maintain the state inside a functional component

```
const root = document.getElementById('root')
// class component
```

```
class Dummy extends React.Component {  
  render() {  
    return <div>this is a class component </div>  
  }  
}  
// here the <Dummy /> is class component  
ReactDOM.createRoot(root).render(<Dummy />)
```

SUNBEAM INFOTECH